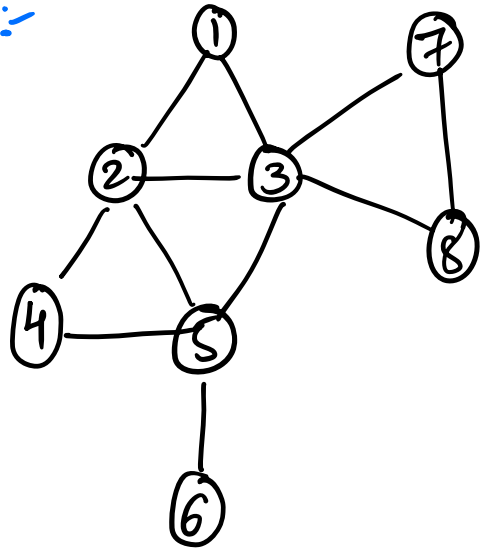
Depth first Search

Idea:

→ To search deeper in the graph whenever Possible.

Example:-



```
DFS(u)
    mark u as "explored"
    For each edge uv incident to u
        if v is not marked "explored" then
            Recursively invoke DFS(v)
        Endif
    Endfor
```

For each node u we have

$u.color$ – color of u

$u.\pi$ – predecessor of u

$u.d$ – discovered time

$u.f$ – finishing time

DFS timestamps each vertex. Each vertex v has two timestamps.

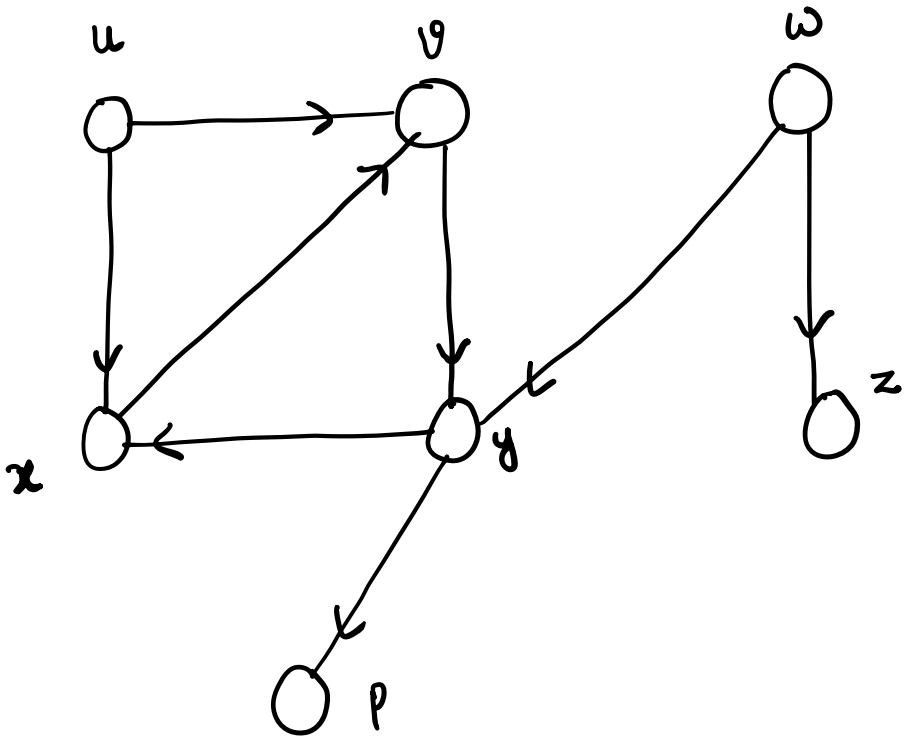
① The first timestamp $v.d$ records when v is first discovered

② The second timestamp $v.f$ record when the search finishes examining v's adjacency list.

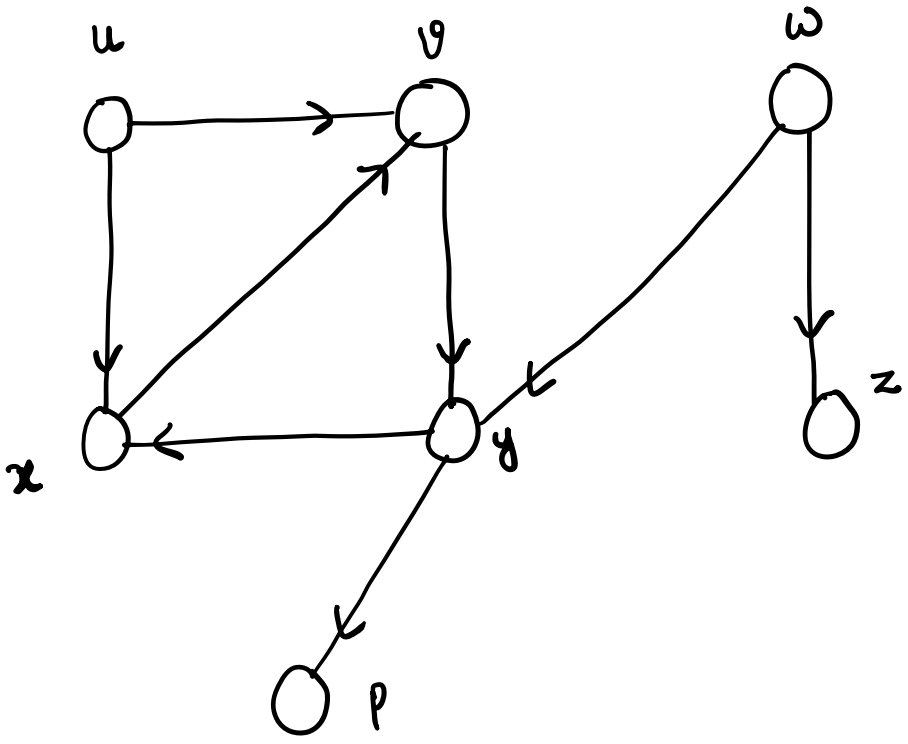
```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)
```

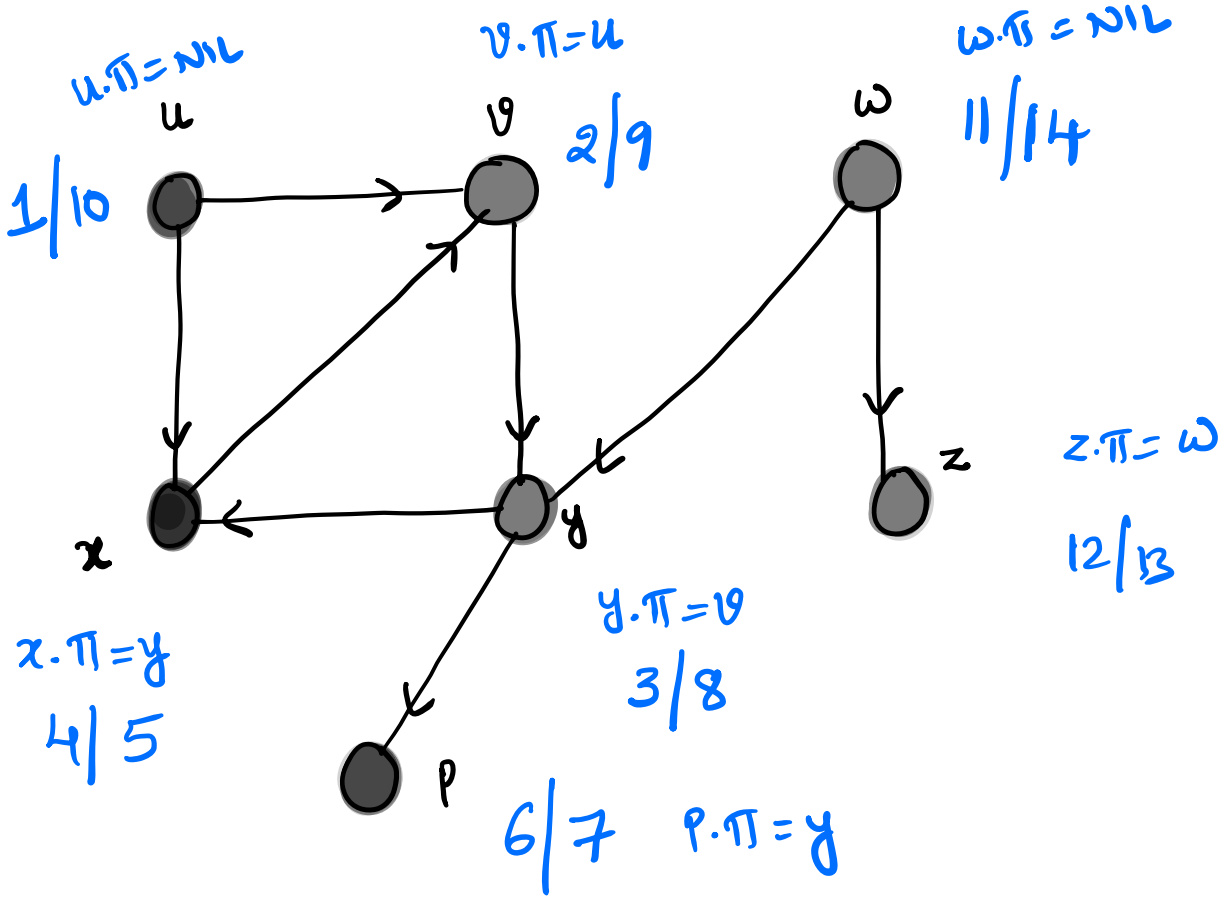
```
DFS-VISIT(G, u)
 1  time = time + 1            // white vertex u has just been discovered
 2  u.d = time
 3  u.color = GRAY
 4  for each v ∈ G.Adj[u]      // explore edge (u, v)
 5      if v.color == WHITE
 6          v.π = u
 7          DFS-VISIT(G, v)
 8  u.color = BLACK            // blacken u; it is finished
 9  time = time + 1
10  u.f = time
```

Example:



Example:


u
v
w
x
y
z
p

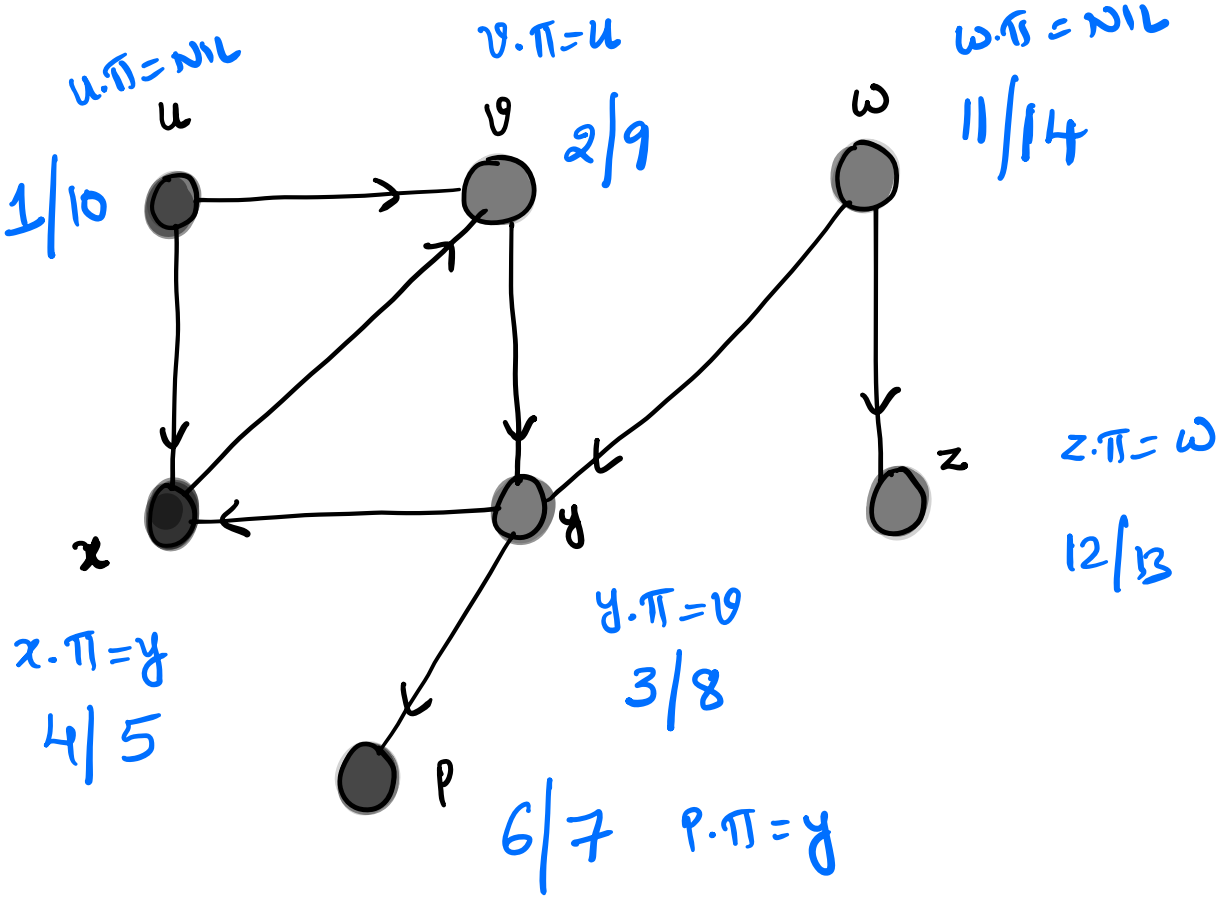

u.π = NIL
1/10
u
v.π = u
2/9
v
w
w.π = NIL
11/14
x
x.π = y
4/5
y
y.π = v
3/8
z
z.π = w
12/13
p
6/7
p.π = y

Depth first Search tree | forest

$$G_\pi = (V, E_\pi)$$

where $E_\pi = \{ (v.\pi, v) \mid v \in V \text{ and } v.\pi \neq NIL \}$

Find the DFS - Forest for the following example.



Classification of edges

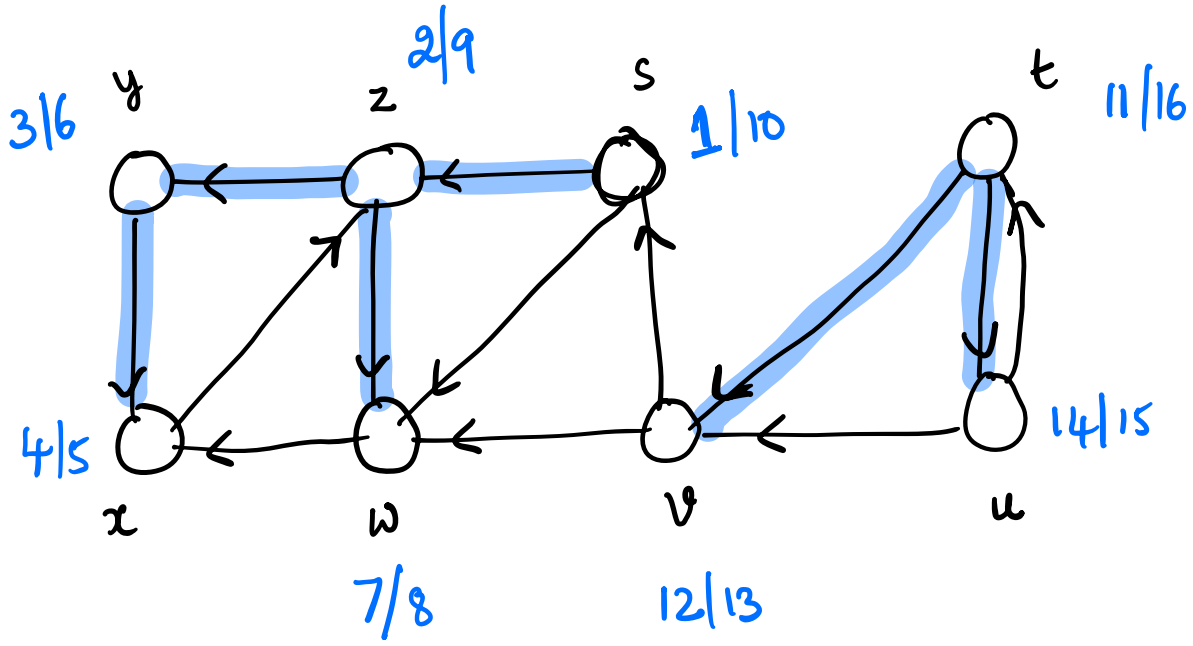
DFS can be used to classify the edges of the input graph.

The type of each edge gives some information about the structure of the graph.

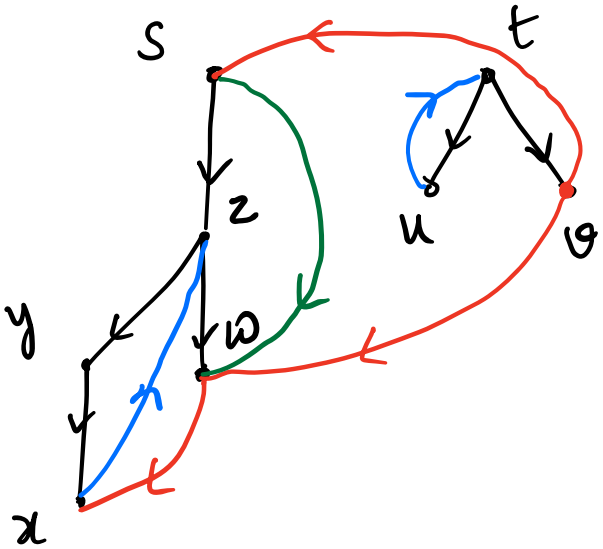
Based on DFS forest G_π we can define four edge types.

1. ***Tree edges*** are edges in the depth-first forest G_π. Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v).
2. ***Back edges*** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.
3. ***Forward edges*** are those nontree edges (u, v) connecting a vertex u to a descendant v in a depth-first tree.
4. ***Cross edges*** are all other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.

Example:



DFS forest:



Tree edges - black colored

Back edges - blue "

Cross edges - red "

forward edges - green "

Q: let uv be a tree-edge then

is there any relation b/w

$u.d$, $u.f$, $v.d$ and $v.f$?

$uv \in E(G)$

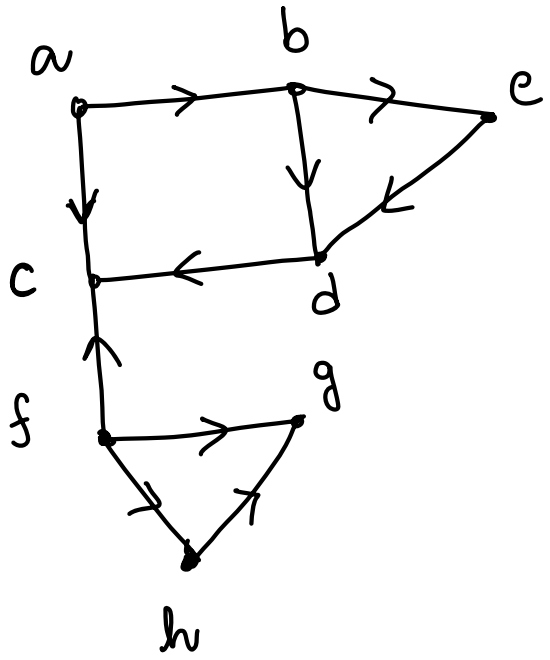
FORWAD EDGE – $u.d < v.d$ & $u.f > v.f$

Back EDGE – $u.d > v.d$ and $u.f < v.f$

Tree edge – $u.d < v.d$ & $u.f > v.f$

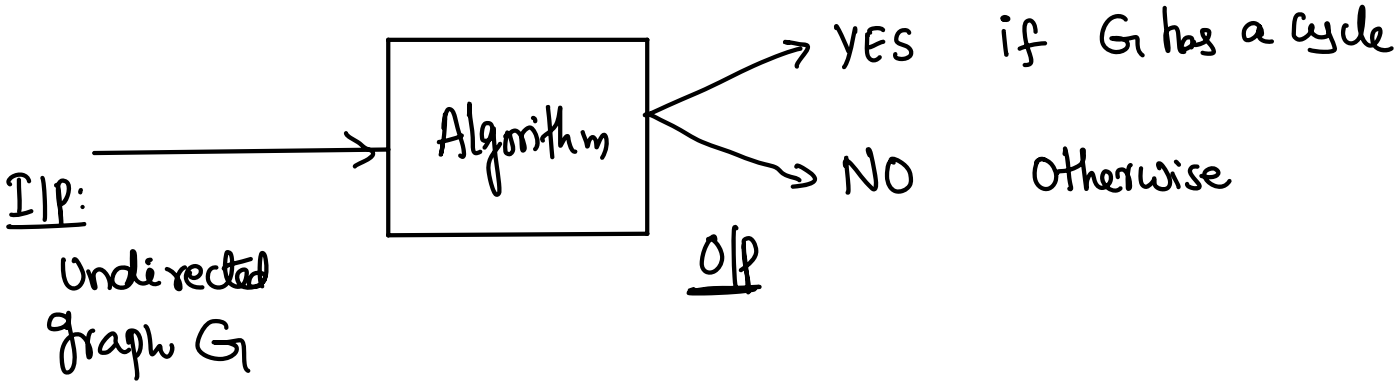
Cross edge – $v.d < v.f < u.d < u.f$

Warmup Questions



- Perform a DFS starting from node 'a', by giving Preference to lower character letters before higher character letters.
- Draw DFS Forest
- Classify the edges

Q: How to test whether a connected undirected graph has a cycle?



True | False : In a DFS search of an undirected graph G, every edge of G is either a tree edge or a back edge.

Notation:

DAG : Directed Acyclic Graph

[Directed graph without cycles]

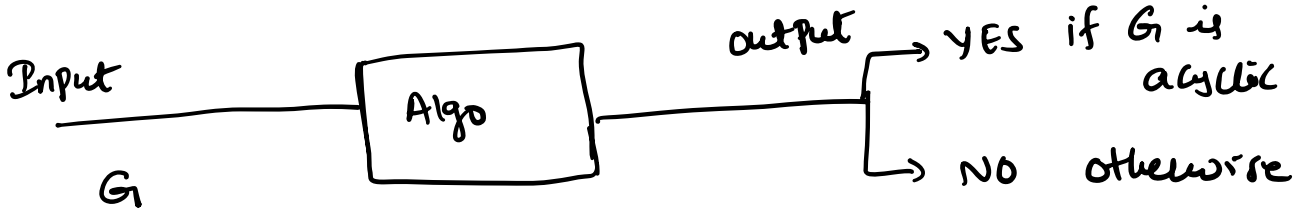
Applications of DFS

1. Testing whether a directed graph has a cycle.

2. Topological Sort.

3. Strongly Connected Components.

Check if a directed graph is acyclic



True/False :

directed

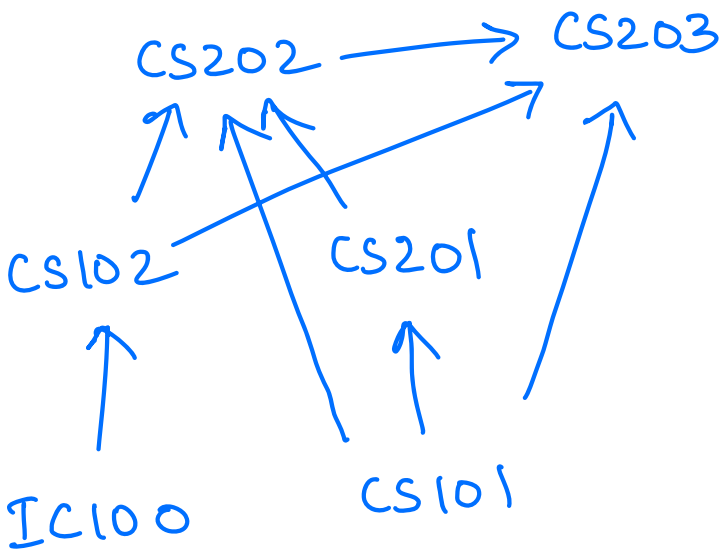
If a DFS of G, Produces a backedge then G has a cycle.

True/False :

If a directed graph G has a cycle then DFS of G produces a back edge.

Lemma: A directed graph is acyclic if and only if a DFS of G yields no back edges.

Topological Sort - Motivation



Prerequisite Graph

Topological Sort

A topological sort of a DAG $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge uv, then u appears before v in the ordering.

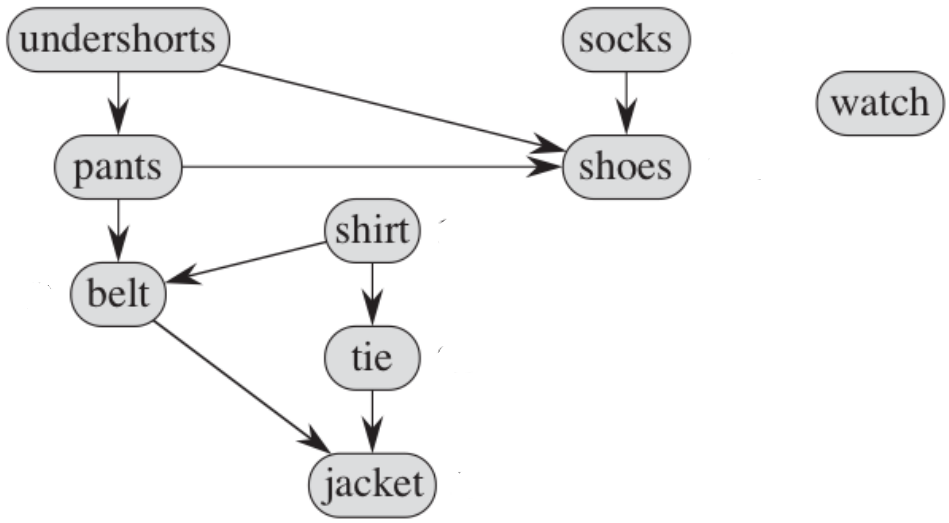


All the edges go from left to right.

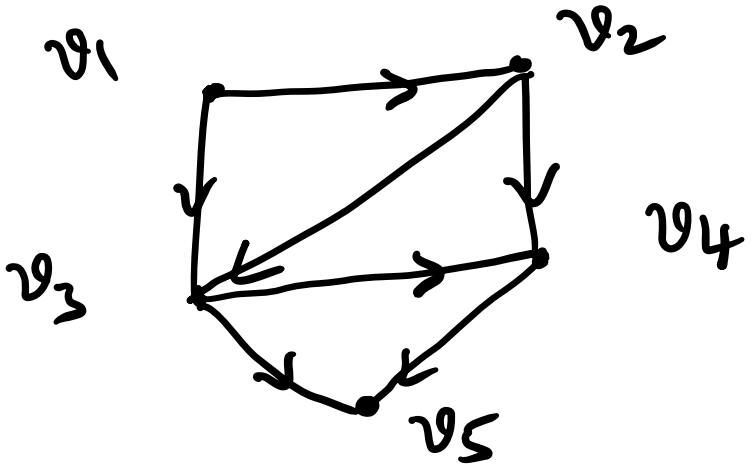
Motivation :

Suppose that you need to Perform many tasks, but some of them cannot begin until certain others are completed.

We can model them using directed graphs


undershorts
socks
watch
pants
shoes
shirt
belt
tie
jacket

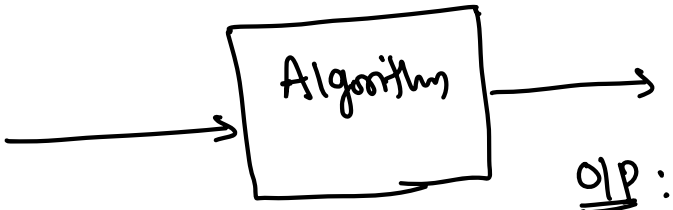
Warmup example



DAG G

Find the topological sort of G.

Topological sort Algorithm



I/P : DAG

O/P : Topological ordering

True|False:

In every DAG G, there is a vertex v with no incoming edges.

True/False:

In every DAG G, there is a vertex v with no ~~incoming~~ outgoing edges.

Q: Does this (above TIF) help to design an algorithm to find topological sort?

What is the Running time?

- Given a DAG $G=(V,E)$, goal is to find a vertex with no incoming edges.

Idea: Using adjacency list, compute in-degree of each vertex by scanning all adjacency lists

This takes $O(V+E)$ time.

Idea: Use DFS to find topological ordering

Algoritum

TOPOLOGICAL-SORT(G)

1 call DFS(G) to compute finishing times $v.f$ for each vertex v
2 as each vertex is finished, insert it onto the front of a linked list
3 **return** the linked list of vertices

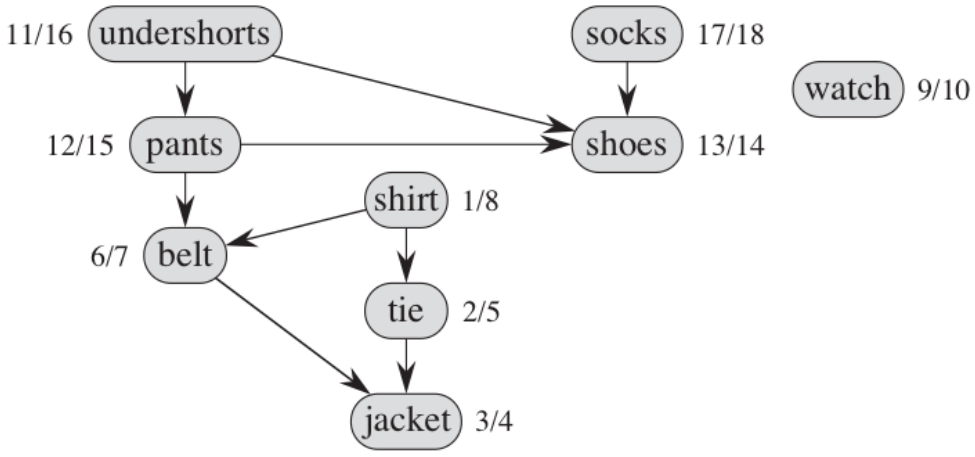
Running time: $O(V+E)$

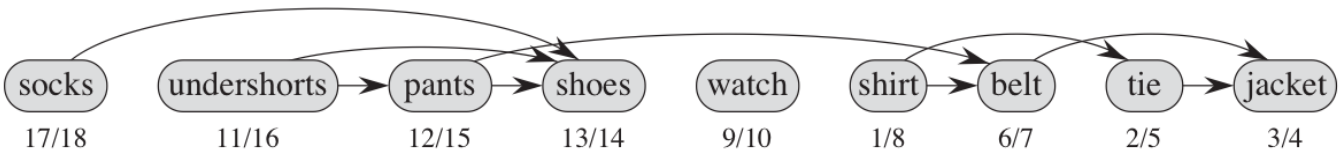
we get the ordering of the vertices of G

$v_1 , v_2 , \ldots v_n$

Given that $v_i.f > v_j.f$ if $j > i$

Example:


11/16
undershorts
socks
17/18
watch
9/10
12/15
pants
shoes
13/14
shirt
1/8
6/7
belt
tie
2/5
jacket
3/4


socks
17/18
undershorts
11/16
pants
12/15
shoes
13/14
watch
9/10
shirt
1/8
belt
6/7
tie
2/5
jacket
3/4